ACTIVIDAD 2: OBJETOS EN JAVASCRIPT

Módulo: Desarrollo Web en Entorno Cliente **Curso:** 2.º DAW **Duración total:** 2 horas **Contenido:** Objetos, propiedades, referencias, métodos, `this`, constructores y encadenamiento opcional.

Objetivos

- Crear y modificar objetos en JavaScript.
- Acceder a las propiedades mediante la notación de punto y de corchetes.
- Utilizar propiedades dinámicas.
- Recorrer las propiedades de un objeto.
- Comprender las diferencias entre referencias y copias.
- Crear métodos que utilicen `this`.
- Construir objetos mediante funciones constructoras.
- Acceder de forma segura a propiedades que pueden no existir.

Organización de la sesión

Actividad	Contenido	Tiempo
1	Creación y manipulación de objetos	15 min
2	Propiedades dinámicas y recorrido de objetos	15 min
3	Referencias, copias y clonación	20 min

Actividad	Contenido	Tiempo
4	Métodos y uso de `this`	20 min
5	Constructores y creación de objetos	25 min
6	Encadenamiento opcional y reto final	25 min
Total		**120 min**

ACTIVIDAD 1. Gestión de un producto

Tiempo estimado: 15 minutos.

Enunciado

Una tienda online necesita almacenar información sobre sus productos. Crea un programa JavaScript que permita gestionar los datos de uno de ellos.

Apartado A. Crear el objeto

Crea un objeto llamado `producto` con las siguientes propiedades:

- `nombre`: `"Teclado mecánico"`.
- `precio`: `79.99`.
- `stock`: `12`.
- `disponible`: `true`.

Muestra por consola el nombre y el precio del producto.

Apartado B. Modificar propiedades

Realiza las siguientes operaciones:

1. Cambia el precio a `69.99`.
2. Reduce el stock a `10`.
3. Añade una propiedad llamada `categoria` con el valor `"Periféricos"`.
4. Añade una propiedad llamada `"fecha de lanzamiento"` con el valor `"2026-09-15"`.
5. Muestra todas las propiedades del objeto.

Apartado C. Acceder y eliminar propiedades

1. Muestra el valor de `"fecha de lanzamiento"` utilizando la notación de corchetes.
2. Crea una variable llamada `propiedad` que almacene el texto `"stock"`.
3. Utiliza esa variable para mostrar el stock.
4. Elimina la propiedad `disponible`.
5. Comprueba que la propiedad se ha eliminado.

Preguntas

1. ¿Por qué no podemos utilizar `producto.fecha de lanzamiento`?
2. ¿Qué diferencia existe entre `producto.stock` y `producto[propiedad]`?
3. ¿Qué devuelve JavaScript cuando intentamos acceder a una propiedad que no existe?

ACTIVIDAD 2. Propiedades dinámicas y recorrido de objetos

Tiempo estimado: 15 minutos.

Enunciado

Una aplicación web permite registrar información de usuarios. Los nombres de algunos campos se reciben dinámicamente desde un formulario.

Apartado A. Crear propiedades dinámicas

Crea las siguientes variables:

```
const campo1 = "nombre";
const campo2 = "email";
const campo3 = "edad";
```

Utilízalas para crear un objeto llamado `usuario` con estos datos:

Campo	Valor
nombre	Ana
email	ana@ejemplo.com
edad	25

Importante: utiliza propiedades calculadas mediante corchetes.

Apartado B. Comparar resultados

Analiza el siguiente código:

```
const campo = "nombre";

const usuario1 = {
    campo: "Ana"
};

const usuario2 = {
```

```
    [campo]: "Ana"
};
```

Responde:

1. ¿Qué propiedades contiene `usuario1`?
2. ¿Qué propiedades contiene `usuario2`?
3. ¿Qué diferencia existe entre `campo` y `[campo]`?
4. ¿Qué mostrarán las siguientes instrucciones?

```
console.log(usuario1.campo);
console.log(usuario1.nombre);

console.log(usuario2.campo);
console.log(usuario2.nombre);
```

Apartado C. Recorrer el objeto

Utiliza un bucle `for...in` para mostrar todas las propiedades y sus valores.

La salida debe tener un formato similar al siguiente:

```
nombre: Ana
email: ana@ejemplo.com
edad: 25
```

Apartado D. Comprobar propiedades

Utiliza el operador `in` para comprobar si el objeto contiene las propiedades `email`, `telefono` y `edad`.

Muestra el resultado de cada comprobación.

ACTIVIDAD 3. Referencias, copias y clonación

Tiempo estimado: 20 minutos.

Enunciado

Vas a trabajar con objetos que almacenan información de usuarios para comprender qué ocurre cuando se asigna un objeto a otra variable.

Apartado A. Analizar las referencias

Ejecuta el siguiente código y explica el resultado:

```
const usuario = {
    nombre: "Carlos",
    edad: 28
};

const administrador = usuario;

administrador.nombre = "Marta";

console.log(usuario.nombre);
console.log(administrador.nombre);
```

Responde:

1. ¿Qué nombre aparece en cada `console.log()`?
2. ¿Se han creado dos objetos diferentes?
3. ¿Por qué modificar `administrador` afecta a `usuario`?
4. ¿Es posible modificar las propiedades de un objeto declarado con `const`?

Apartado B. Crear una copia independiente

Partiendo del siguiente objeto:

```
const producto = {
    nombre: "Ratón inalámbrico",
    precio: 29.99,
    stock: 15
};
```

Crea un objeto llamado `copiaProducto` utilizando `Object.assign()`.

Después:

1. Cambia el precio de `copiaProducto` a `24.99`.
2. Cambia el stock de `copiaProducto` a `20`.
3. Muestra ambos objetos.
4. Comprueba si las modificaciones de la copia afectan al original.

Apartado C. Objetos anidados

Analiza el siguiente código:

```
const usuario1 = {
    nombre: "Lucía",
    direccion: {
        ciudad: "Valencia",
        codigoPostal: "46001"
    }
};

const usuario2 = Object.assign({}, usuario1);

usuario2.nombre = "Laura";
usuario2.direccion.ciudad = "Castellón";

console.log(usuario1.nombre);
console.log(usuario1.direccion.ciudad);

console.log(usuario2.nombre);
console.log(usuario2.direccion.ciudad);
```

Responde:

1. ¿Qué ocurre al modificar `usuario2.nombre`?
2. ¿Qué ocurre al modificar `usuario2.direccion.ciudad`?
3. ¿Por qué se producen resultados diferentes?
4. ¿Qué método puedes utilizar para realizar una clonación profunda de este objeto?

Apartado D. Aplicar la solución

Utiliza `structuredClone()` para crear una copia independiente de `usuario1`.

Modifica la ciudad de la copia y comprueba que la ciudad del objeto original no cambia.

ACTIVIDAD 4. Métodos y uso de `this`

Tiempo estimado: 20 minutos.

Enunciado

Una aplicación de gestión de usuarios necesita que cada objeto pueda mostrar sus propios datos y modificarlos mediante métodos.

Apartado A. Crear un objeto con métodos

Crea un objeto llamado `cuenta` con las siguientes propiedades:

- `titular`: `"María"`.
- `saldo`: `500`.
- `moneda`: `"EUR"`.

Añade los siguientes métodos:

- `mostrarSaldo()`: muestra el titular y el saldo.
- `ingresar(cantidad)`: aumenta el saldo en la cantidad indicada.
- `retirar(cantidad)`: reduce el saldo si hay dinero suficiente.
- `mostrarDatos()`: muestra toda la información de la cuenta.

Utiliza `this` para acceder a las propiedades del objeto.

Apartado B. Probar los métodos

Ejecuta las siguientes operaciones:

```
cuenta.mostrarSaldo();

cuenta.ingresar(150);

cuenta.retirar(80);

cuenta.mostrarDatos();

cuenta.retirar(1000);

cuenta.mostrarSaldo();
```

El método `retirar()` debe mostrar un mensaje si no hay saldo suficiente.

Apartado C. Comprender `this`

Analiza el siguiente ejemplo:

```
const usuario = {
    nombre: "Pedro",

    saludar() {
        console.log("Hola, soy " + this.nombre);
    }
};

const administrador = {
    nombre: "Elena",
    saludar: usuario.saludar
};

usuario.saludar();
administrador.saludar();
```

Responde:

1. ¿Qué muestra la primera llamada?
2. ¿Qué muestra la segunda llamada?
3. ¿Por qué el método puede mostrar nombres diferentes si la función es la misma?
4. ¿Qué papel desempeña `this`?

Apartado D. Ampliación

Añade a `cuenta` un método llamado `transferir(otraCuenta, cantidad)`.

El método debe:

1. Comprobar si el saldo es suficiente.
2. Restar la cantidad a la cuenta actual.
3. Añadir la cantidad al saldo de la otra cuenta.
4. Mostrar un mensaje indicando si la transferencia se ha realizado.

Crea una segunda cuenta para comprobar su funcionamiento.

ACTIVIDAD 5. Constructores de objetos

Tiempo estimado: 25 minutos.

Enunciado

Una tienda necesita registrar muchos productos. En lugar de crear cada objeto manualmente, vas a utilizar una función constructora.

Apartado A. Crear el constructor

Define una función constructora llamada `Producto` que reciba:

- `nombre`.
- `precio`.
- `stock`.

La función debe almacenar estos datos en el objeto mediante `this`.

Apartado B. Crear métodos

Añade al constructor los siguientes métodos:

- `mostrarInformacion()` : muestra el nombre, el precio y el stock.
- `actualizarPrecio(nuevoPrecio)` : modifica el precio.
- `vender(cantidad)` : reduce el stock si hay unidades suficientes.

Apartado C. Crear instancias

Crea tres productos diferentes:

```
const producto1 = new Producto("Portátil", 899.99, 5);
const producto2 = new Producto("Monitor", 199.99, 10);
const producto3 = new Producto("Teclado", 49.99, 20);
```

Comprueba que cada objeto tiene sus propios datos.

Apartado D. Realizar operaciones

1. Muestra la información de los tres productos.
2. Actualiza el precio del portátil a `849.99`.
3. Vende dos unidades del monitor.
4. Intenta vender quince unidades del teclado.
5. Muestra de nuevo la información de todos los productos.

Apartado E. Comprobar instancias

Investiga las siguientes expresiones:

```
console.log(producto1 instanceof Producto);

console.log(producto2 instanceof Producto);

console.log(producto1 === producto2);
```

Explica qué significa cada resultado.

Apartado F. Prototipos

Ejecuta las siguientes instrucciones:

```
console.log(Object.getPrototypeOf(producto1) === Producto.prototype);

console.log(Object.getPrototypeOf(producto2) === Producto.prototype);
```

Responde:

1. ¿Qué relación existe entre las instancias y `Producto.prototype`?
2. ¿Qué es un prototipo?
3. ¿Qué permite la cadena de prototipos en JavaScript?

ACTIVIDAD 6. Encadenamiento opcional y reto final

Tiempo estimado: 25 minutos.

Parte A. Acceso seguro a propiedades

Tiempo estimado: 10 minutos.

Una aplicación recibe información de clientes procedente de una API. Algunos clientes no tienen todos los datos registrados.

Utiliza el siguiente objeto:

```
const cliente1 = {
    nombre: "Sergio",
    contacto: {
        email: "sergio@ejemplo.com",
        telefono: "600123123"
    }
};

const cliente2 = {
    nombre: "Paula"
};

const cliente3 = null;
```

Ejercicios

1. Acceso normal

Escribe una instrucción para mostrar el correo electrónico de `cliente1`.

2. Acceso a una propiedad inexistente

Intenta obtener el correo electrónico de `cliente2` utilizando la notación de punto.

¿Qué sucede?

3. Encadenamiento opcional

Utiliza `?.` para acceder al correo electrónico de `cliente2` sin provocar un error.

Guarda el resultado en una variable llamada `email2`.

4. Cliente nulo

Utiliza el encadenamiento opcional para intentar obtener el nombre de `cliente3`.

¿Qué devuelve la expresión?

5. Valor alternativo

Utiliza el operador `??` para mostrar `"Correo no disponible"` cuando el correo electrónico no exista.

6. Comparación

Explica la diferencia entre estas dos expresiones:

```
cliente2.contacto?.email
```

```
cliente2.contacto.email
```

Parte B. Reto final: gestión de clientes

Tiempo estimado: 15 minutos.

Desarrolla un pequeño programa de gestión de clientes que utilice los contenidos estudiados.

Requisitos

1. Crear el constructor `Cliente`

Debe recibir los siguientes parámetros:

- `nombre`.
- `email`.
- `telefono`.

Al crear cada cliente, almacena sus datos en las propiedades correspondientes.

2. Añadir métodos

Implementa los siguientes métodos:

- `mostrarDatos()`: muestra los datos del cliente.
- `actualizarEmail(nuevoEmail)`: actualiza el correo electrónico.
- `tieneTelefono()`: devuelve `true` si existe un teléfono válido y `false` en caso contrario.

3. Crear clientes

Crea tres clientes diferentes. Uno de ellos debe tener el teléfono vacío.

4. Crear un objeto de configuración dinámica

Crea un objeto llamado `configuracion` con una propiedad cuyo nombre se construya a partir de una variable.

Por ejemplo, si la variable contiene `"modo"`, la propiedad deberá llamarse `modo`.

5. Crear una copia

Crea una copia independiente de uno de los clientes utilizando `Object.assign()`.

Modifica el correo electrónico de la copia y comprueba que el correo original no cambia.

6. Acceder de forma segura

Utiliza el encadenamiento opcional para mostrar el teléfono de un cliente sin provocar errores cuando no exista.

7. Mostrar resultados

Muestra por consola:

- Los datos de los tres clientes.
- El resultado de `tieneTelefono()` para cada uno.
- Los datos originales y los modificados de la copia.
- El valor de la propiedad dinámica de `configuracion`.

Preguntas finales

1. ¿Qué ventajas tiene utilizar una función constructora frente a crear manualmente cada objeto?
2. ¿Qué diferencia hay entre copiar un objeto y copiar su referencia?
3. ¿Para qué sirve `this`?
4. ¿Qué ventaja ofrece el encadenamiento opcional?
5. ¿Qué diferencia hay entre una propiedad normal y una propiedad calculada?

Entrega

Entrega un archivo JavaScript con la solución de las seis actividades.

El código debe estar organizado por apartados y utilizar nombres de variables descriptivos.

Se valorará especialmente:

- Que el código funcione correctamente.
- Que se utilicen adecuadamente los objetos y sus propiedades.
- Que los métodos utilicen `this` cuando corresponda.
- Que se comprenda la diferencia entre referencias y copias.
- Que se utilicen correctamente los constructores y el encadenamiento opcional.
- Que el código sea claro, esté ordenado y contenga comentarios cuando sean necesarios.



Fecha de creación: 27 d'octubre del 2026
Última actualización: 27 d'octubre del 2026